Software Requirements and Design Document

For

Group <20>

Version 1.0

Authors:
Alex Fenoga
Alejandro Valdes
Brian Nelson
Caleb Dindinger
Daniel Smith

1. Overview (5 points)

An online, web-based bartering application inspired by item trading systems in games such as CSGO. Users fill up their "inventory" by uploading photos and item descriptions for things they are willing to trade. They are then able to create a bid listing for any item in their inventory. Bid listings are displayed publicly for all other users to see. Other users are able to bid on publicly listed items using one or multiple items from their own inventory. After a specified amount of time, the bid listing owner can look through all the bids, selecting the best one. The user who's bid got selected gets a notification/message from the listing owner with information on how the items are to be exchanged.

2. Functional Requirements (10 points)

Users - High

- [1] Create an account using username and password.
- [2] Log into created account using username and password.
- [3] Add items (image + description) to their inventory.
- [4] Delete items from their inventory.
- [5] List items from inventory for public bidding.
- [6] Remove listed items from public bidding.
- [7] Look at all the bids made by other users.
- [8] Select the best bid, ending the listing.
- [9] Place bids, with items from their inventory, on public listings.
- [10] Remove their bid.
- [26] Set a custom message that will be sent to the winning bidder in a public listing.

Users - Medium

- [11] Add tags to items in inventory to indicate the item's type.
- [12] Filter public listings by item types.
- [13] Restrict valid bids to specific item types.
- [14] Add location to listing.
- [15] Filter public listings by location.

Server - High

- [17] Create a new user account.
- [18] Authenticate a user trying to log in.
- [19] Create a new user item entry, provided an image file and a description.
- [20] Remove a specific user item entry.
- [21] Retrieve all user item entries.
- [22] Create a public listing entry for a specific item.
- [23] Delete a public listing entry.
- [24] Retrieve all public listings.
- [25] Send a custom message (set by listing owner) to the user who made the winning bid.

3. Non-functional Requirements (10 points)

NFR.1: User passwords must be stored as a hash + salt using PBKDF2 / HMAC-SHA512.

NFR.2: All transactions must be atomic and guarantee data integrity.

NFR.3: The web application must support the latest versions of Chrome and Firefox.

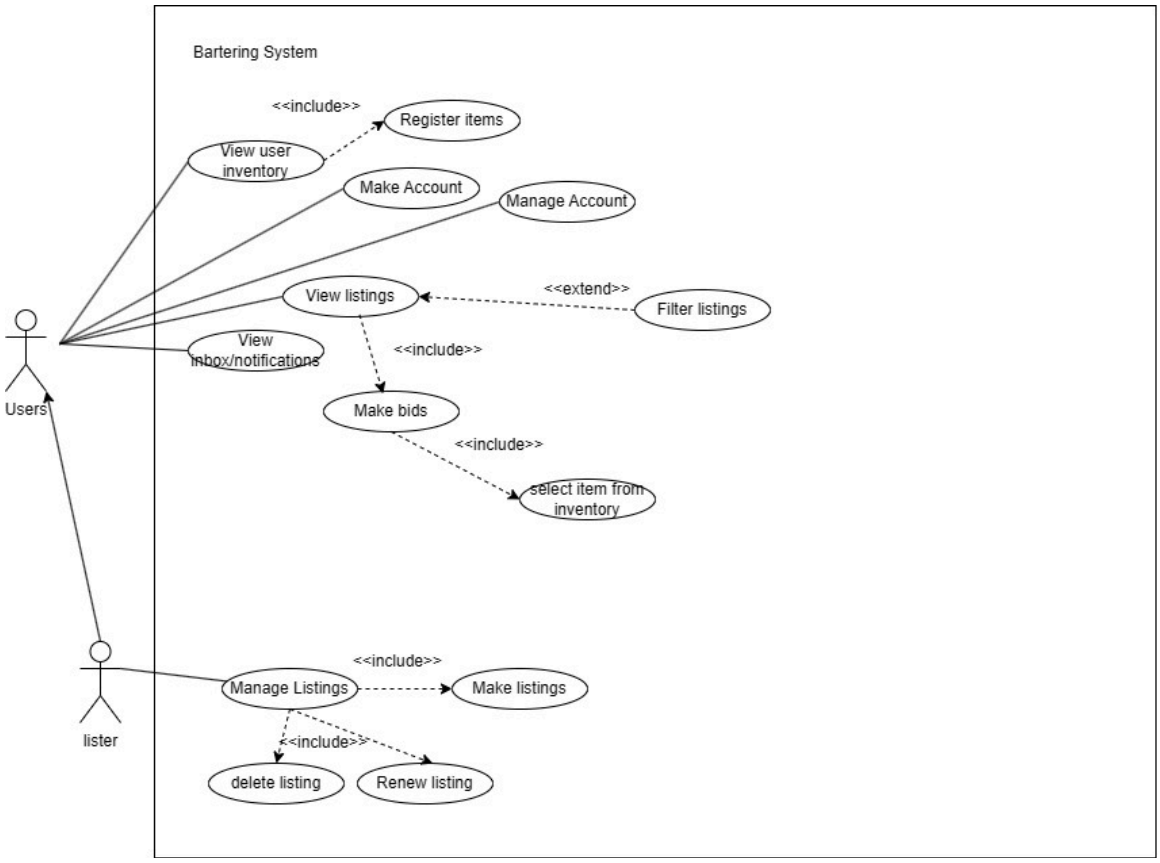
NFR.4: The system shall process and respond to user registration requests within 500ms.

NFR.5: The system shall process and respond to user login requests within 500ms.

NFR.6: The system shall respond to requests for a list of 100 offers within 300ms.

4. Use Case Diagram (10 points)

The image below is the Use Case diagram for Increment 1. Textual descriptions are not included as they are not required for the first iteration. The diagram is also located under the *documents/diagrams/* folder in the repository where it is stored as a *.jpeg* and *.drawio* file.



Make Account Specification
1. Name: Make Account
2. Description: This use case describes how a user creates an account so that they can use the application.
3. Participating Actors: User
4. Preconditions:
   - The User is not currently logged into an account
   - The username must not already exist
   - The password must meet requirements (e.g. be of sufficient length)
5. Postconditions:
   - The account was created
   - The User is authenticated
6. Flow:
   1. The User enters a username and password into the form
   2. The User submits the form
   3. The system verifies that the username is not taken
   4. The system verifies that the password meets requirements

   5. The system persists the user data and the hashed and salted password
   6. The system displays a view confirming that registration was successful
7. Alternative Flows:
   - Username already exists in step 3
     - The system asks the user to enter a different username
     - The use case resumes at step 1
   - The password does not meet requirements in step 4
     - The system informs the user of the password requirements
     - The use case resumes at step 1

Register Items Specification
1. Name: Register Items
2. Description: This use case describes how a user adds an item to their inventory.
3. Participating Actors: User
4. Preconditions:
   - The User is logged in
5. Postconditions:
   - The item was created
6. Flow:
   1. The User opened the item creation form
   2. The User enters the item's name, a description about it, and uploads an image of the item
   3. The User submits the form
   4. The system verifies that the form is filled out completely
   5. The system persists the item data
   6. The system displays a confirmation that the item creation was successful
   7. The system returns the User to the inventory view
7. Alternative Flows:
   - Data is found to be missing in step 4
     - The system asks the user to enter the required information
     - The use case resumes at step 2

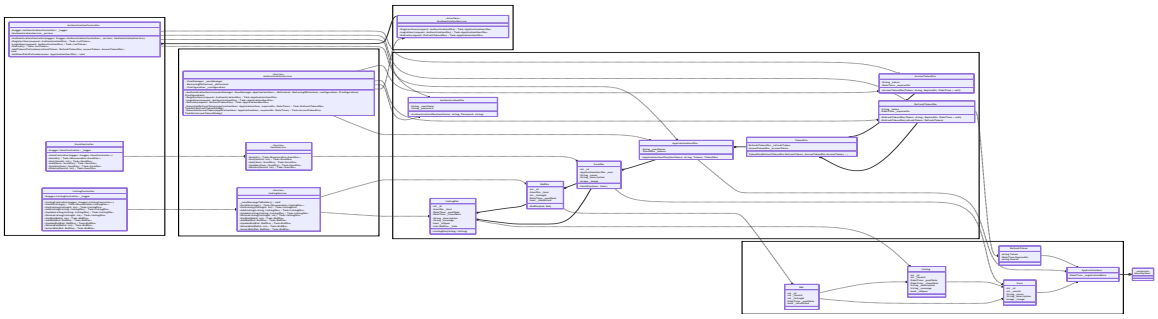
View User Inventory Specification
1. Name: View User Inventory
2. Description: This use case describes how a user views their inventory.
3. Participating Actors: User
4. Included Use Cases: Register Items
5. Preconditions:
   - The User is logged in
   - The User is on a page that lets them navigate to their inventory
6. Postconditions:
   - The User can see the items that they own
7. Flow:
   1. The User clicks the Inventory button
   2. The system retrieves the User's items
   3. The system displays the inventory page with the User's items
8. Alternative Flows:
   - The User clicks the Add Item button on the inventory page
     - The system triggers the Register Items use case.

5. Class Diagram and/or Sequence Diagrams (15 points)

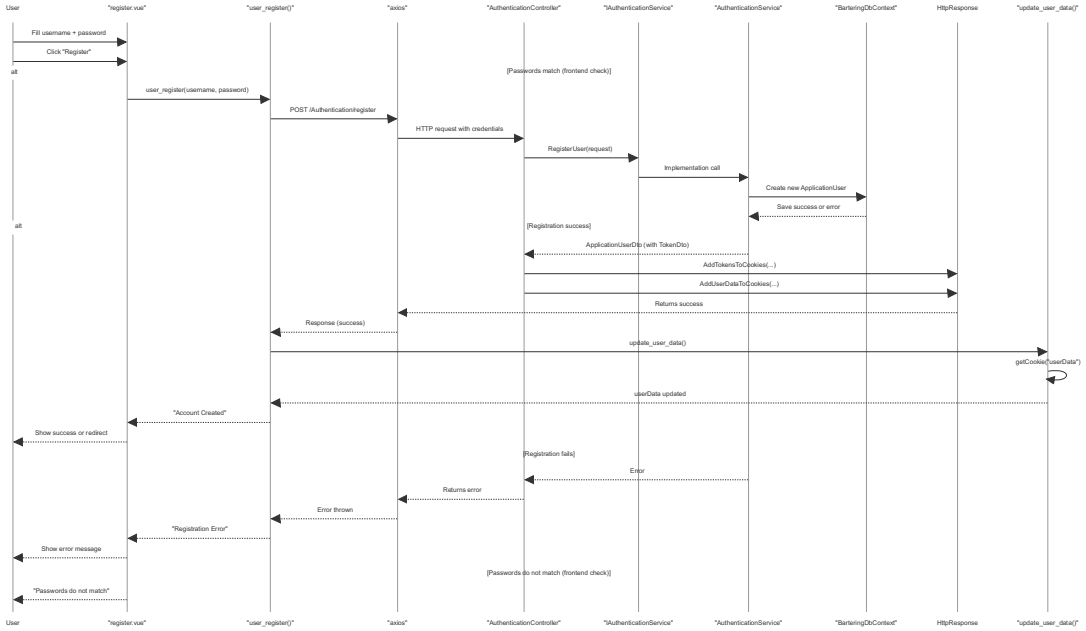
Please see the class diagram (and sequence diagrams) under documents/diagrams/ in the repository. There is both a markdown and a svg file. The diagram was made using Mermaid.js, hence the markdown file as well as some ‘oddities’ in the diagram that would require manual modification or recreation through a different tool to improve. The svg file has a different layout compared to what you would see in the markdown render on GitHub as the layout of the export

was set to use something that GitHub does not support. The actual details of the two *should* be the same, but the markdown file is the source. You can look at whichever one you feel is easier to review.

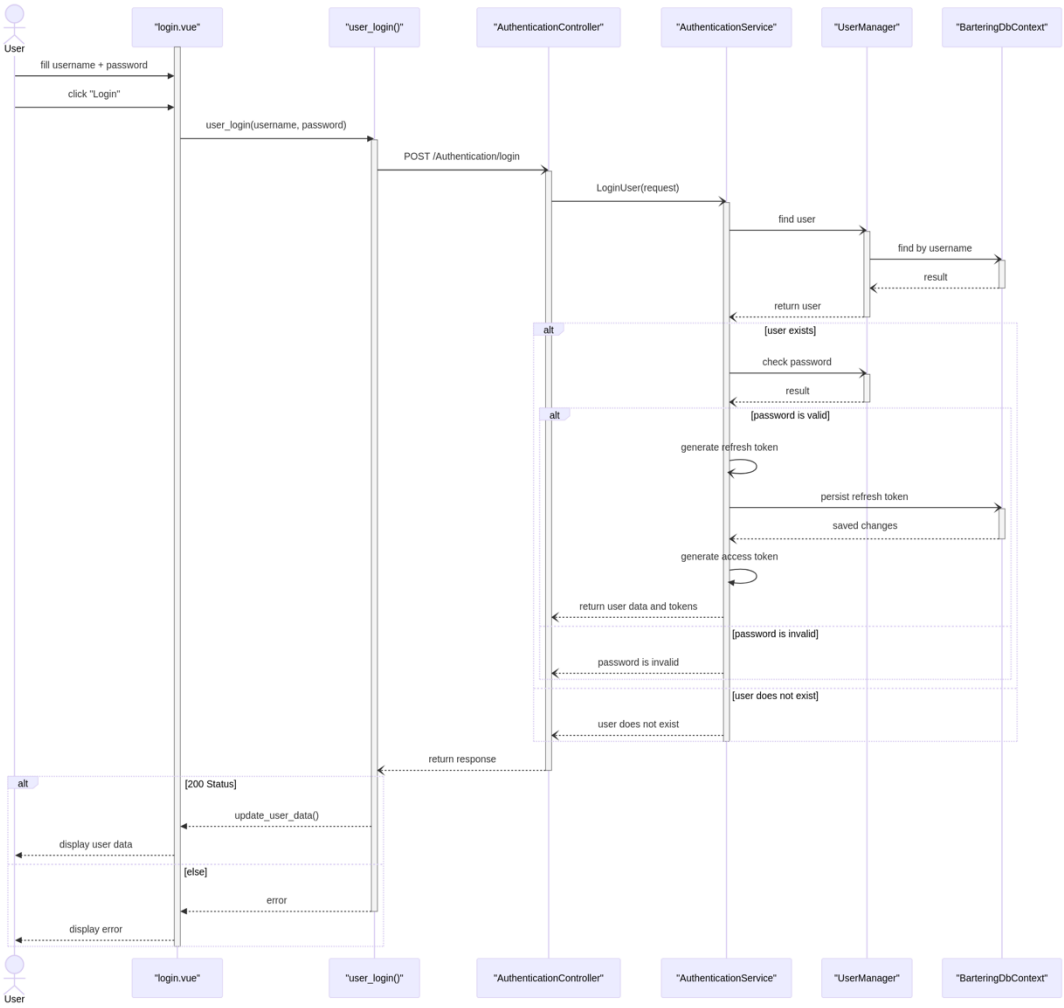
It is currently not really feasible to fit the class diagram in this document and it seems some details, such as multiplicity and some lines, are actually lost when the svg is inserted into Word (using a png had worse problems). A copy of the .svg is included here anyways for completion.



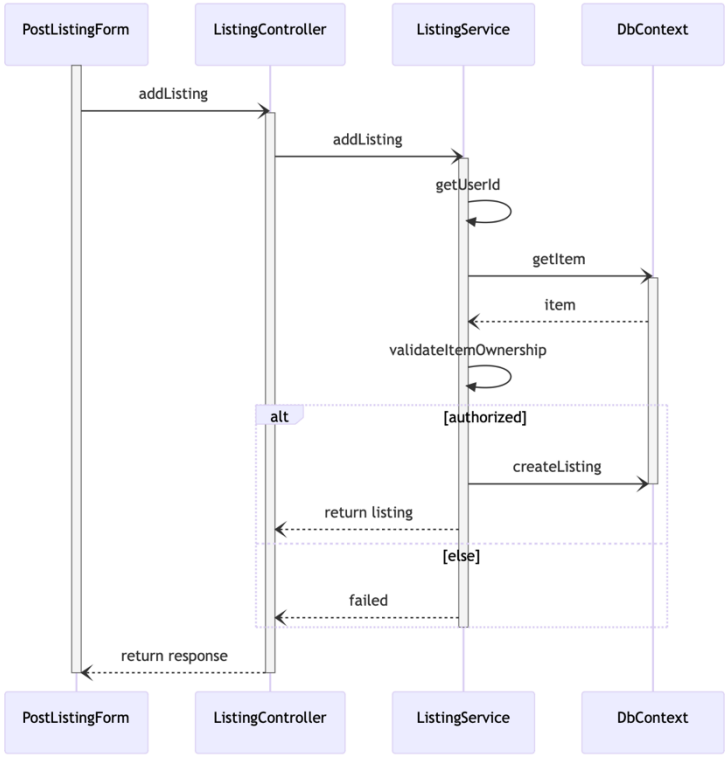
Registration Sequence



Login Sequence



Post Listing Sequence



6. Operating Environment (5 points)

This will be a web-based application that should run on all modern browsers with native support for ES2016. Being a web-app, there is little dependency on operating system and hardware specifications of the user's computer. It should also function on smart phones, with some potential UI changes made to optimize user experience. Aside from a relatively modern browser, all which is required is an internet connection.

7. Assumptions and Dependencies (5 points)

As of this increment (1), we do no plan on using any external third-party or commercial components such as Firebase. Our desire is to make everything ourselves with the use of minimal tooling and frameworks such as VueJS. That being said, most of the team may have experience with front-end or back-end development, but using other languages or frameworks. The assumption here is that team members will spend time familiarizing themselves with VueJS3 and C# so that they can make valuable contributions to the project.